

Module 3: Functions & Interfaces

1. Introduction to Functions in TypeScript

Functions in TypeScript allow defining reusable blocks of code with type safety.

2. Function Declaration & Return Types

```
typescript  
  
function greet(name: string): string {  
  return "Hello, " + name;  
}  
  
console.log(greet("Alice"));
```

3. Optional and Default Parameters

Optional Parameters:

```
typescript  
  
function greet(name: string, age?: number): string {  
  return age ? `Hello, ${name}, age ${age}` : `Hello, ${name}`;  
}  
  
console.log(greet("Alice"));  
console.log(greet("Bob", 25));
```

Default Parameters:

```
typescript  
  
function greet(name: string, age: number = 18): string {  
  return `Hello, ${name}, age ${age}`;  
}  
  
console.log(greet("Alice"));  
console.log(greet("Bob", 25));
```

4. Rest Parameters

```
typescript
```

```
function sum(...numbers: number[]): number {  
  return numbers.reduce((acc, num) => acc + num, 0);  
}  
console.log(sum(1, 2, 3, 4, 5));
```

5. Function Types & Call Signatures

```
typescript  
  
type MathOperation = (a: number, b: number) => number;  
let add: MathOperation = (x, y) => x + y;  
console.log(add(5, 10));
```

6. Introduction to Interfaces

Interfaces define the structure of an object.

```
typescript  
  
interface Person {  
  name: string;  
  age: number;  
  greet(): string;  
}  
  
let user: Person = {  
  name: "Alice",  
  age: 25,  
  greet() {  
    return `Hello, my name is ${this.name}`;  
  }  
};  
  
console.log(user.greet());
```

7. Extending Interfaces

Interfaces can extend other interfaces.

```
typescript  
  
interface Animal {  
  name: string;
```

```
sound(): string;
}
interface Dog extends Animal {
breed: string;
}
let myDog: Dog = {
name: "Buddy",
breed: "Labrador",
sound() {
return "Woof!";
}
};
console.log(myDog.sound());
```

8. Function Interfaces

```
typescript
interface MathFunc {
(x: number, y: number): number;
}
let multiply: MathFunc = (a, b) => a * b;
console.log(multiply(5, 3));
```

Practice Questions:

1. Define a function `subtract` that takes two numbers and returns their difference.
2. Modify the `sum` function to accept only even numbers.
3. Create an interface `Car` with properties `brand`, `year`, and a method `start()`.
4. Extend the `Car` interface to include `ElectricCar` with a `batteryLife` property.
5. Write a function that implements a function interface to calculate the area of a rectangle.

Practice Answers:

1. Function `subtract`:

```
typescript
function subtract(a: number, b: number): number {
return a - b;
}
```

2. Modified `sum` function:

typescript

```
function sum(...numbers: number[]): number {  
  return numbers.filter(num => num % 2 === 0).reduce((acc, num) => acc + num, 0);  
}
```

3. `Car` Interface:

typescript

```
interface Car {  
  brand: string;  
  year: number;  
  start(): string;  
}
```

4. `ElectricCar` Interface:

typescript

```
interface ElectricCar extends Car {  
  batteryLife: number;  
}
```

5. Function interface for rectangle area:

typescript

```
interface AreaFunc {  
  (width: number, height: number): number;  
}  
  
let rectangleArea: AreaFunc = (w, h) => w * h;  
console.log(rectangleArea(5, 10));
```